Előadói jegyzet - ConfigMap, Secret, External Secrets Operator

Ötletek és kulcsmondatok az egész előadáshoz.

---

1. rész: ConfigMap

Slide: Mi az a ConfigMap?

- Vezesd be úgy, mint a "K8s natív .env fájl"-ját.
- Hangsúlyozd: **namespace-scoped** - nem lehet két azonos nevű ConfigMap egy namespace-ben, de különbözőkben igen.
- Az **1 MiB** korlát az etcd korlátozása, nem véletlen.
- Anekdóta: sokan idegenül azzal kezdenek, hogy "rakjunk titkokat is ide" - a következő dia épp erről szól.

Slide: Milyen problémát old meg?

- Kösd a 12-factor app III. alapelvéhez (előző dián már bevezetve).
- Mondd ki a "build once, run anywhere" mantrát.
- Konkrét példa: ugyanaz az image fut dev-en `LOG_LEVEL=debug`-gel, prod-on `info`-val.

Slide: Előnyök / Hátrányok

- A "Hátrányok" oszlop fontosabb mint az "Előnyök".
- Hangsúlyozd a **"érzékeny adat soha"** részt - ez vezet át a Secret-re.
- Anekdóta: "valaki a csapatban biztosan rakott már jelszót ConfigMap-be" - ez normális, ezért beszélünk róla.

Slide: Hogyan működik belül?

- A két csatolási mód a **legfontosabb** koncepció ebben a részben.
- Vizuális segítség: rajzolj a tábla/levegőbe egy nyilat: "indítás → env" vs "folyamatos → volume".
- A 30-60 mp-es szinkron a kubelet syncFrequency beállítása.

Slide: ConfigMap létrehozása

- Mutasd meg, hogy a CLI parancs **YAML-t generál a háttérben** (`-o yaml --dry-run=client`).
- Kérdezd meg: "Ki használja CLI-ből, ki YAML-ből?" - jellemzően YAML nyer GitOps miatt.

Slide: Pod-hoz csatolás - env változó

- Az `envFrom` az összes kulcsot env-ként kihúzza, ezt érdemes kiemelni.
- Alternatíva: `env: -name: -valueFrom: configMapKeyRef:` egy konkrét kulcshoz.
- A **figyelem szöveg** a kulcs - mindenki ezen csúszik el először.

Slide: Pod-hoz csatolás - volume mount

- A trükk: minden kulcs **egy fájlként** jelenik meg a mountPath alatt.
- A `subPath` használata szétlövi a hot reload-ot - ezt sokan nem tudják.
- Vizuális: `ls /etc/config` és `cat /etc/config/LOG_LEVEL` egy valós példán.

Slide: Csatolási módok összehasonlítása

- A táblázat tömör összefoglaló - **ne olvasd fel**, hadd nézzék.
- Egy mondat: "ha hot reload kell, csak a volume mount működik, az is csak subPath nélkül".

Slide: Biztonság és limitációk

- Az **etcd plaintext** rész a legfontosabb. Ezt vidd át a Secret szekcióba is.
- Az immutable flag használati esete: ha tudod, hogy nem fog változni, performancia nyereség.

Slide: Alternatívák

- Csak felsorolás-szinten - ne menj mélyre.
- Ha kérdezik a Helm vs Kustomize-t: "Helm = template engine, Kustomize = patch overlay" - ennyi elég.

Slide: DEMO 1

- **Egyetlen fő üzenet:** env-pod nem frissül, volume-pod frissül.
- A demo során **ne magyarázd a YAML-t** sorról sorra - csak a `kubectl apply` és `kubectl exec` parancsokat.
- Ha jut idő: mutasd meg `kubectl describe pod`-dal hogyan jelenik meg a volume.

Időzítés - 1. rész

- ~10-12 perc az egész szekció (3 perc demo + 7-9 perc dia).
- Ha csúszol: a "Hogyan működik belül?" és "Alternatívák" diákat lehet röviden átfutni.
- Ha gyors vagy: bemutathatod a `kubectl edit configmap`-et live.

---

2. rész: Secret

Slide: Mi az a Secret?

- Vezesd be úgy, mint a **ConfigMap testvérét, csak érzékeny adatokra**.
- Az "Opaque" a default - 99%-ban ezt látod a gyakorlatban.
- A többi típus (`tls`, `dockerconfigjson`) **speciális cél** - tooling érti.

Slide: Milyen problémát old meg?

- Történelmi ív: "régen ConfigMap-be rakták a jelszót, mert csak az volt".
- A Secret nem azért biztonságosabb, mert titkosított, hanem mert **a tooling máshogy bánik vele** (tmpfs, RBAC, audit).

Slide: Előnyök / Hátrányok

- Az "éles környezetben kevés" mondat **átvezeti** a 3. szekcióhoz.
- Példa: "dev-ben elég, de ha 5 csapat / 3 cluster / GDPR audit jön, már nem".

Slide: A nagy félreértés - base64 ≠ titkosítás

- **Ez a szekció legfontosabb diája.** Itt áll meg az óra.
- Mondd ki: "a base64 csak kódolás, NEM titkosítás. Bárki visszafejti."
- A demo során élőben végigjátsszuk - ezért most csak elvet beszélünk.
- Anekdóta: cégek, akik szerepeltek hírekben, mert "a Secret-ek base64-be vannak" - ez nem védelem.

Slide: Hogyan működik belül?

- Két nem nyilvánvaló dolog:
  1. Az etcd alapból NEM titkosítja - külön be kell kapcsolni (encryption at rest).
  2. A pod-on **tmpfs**-re mountolódik (RAM, nem disk) - kis biztonsági plusz.
- A csatolási módok ugyanazok mint ConfigMap-nél - ezt csak gyorsan említsd.

Slide: Secret létrehozása

- A YAML példa **base64-kódolt** értéket vár - ezt sokan elfelejtik.
- Tipp: `echo -n "jelszó" | base64` - a `-n` fontos! (nélküle plusz `\n`).
- A CLI verzió ezt megcsinálja helyetted.

Slide: Secret típusok

- A táblázat csak **referencia** - ne olvasd fel.
- Hangsúlyozd: a `kubernetes.io/tls` **kötött struktúra** (`tls.crt` + `tls.key`).
- A `dockerconfigjson` az `imagePullSecrets`-hez kell - private registry esetén.

Slide: Biztonság és limitációk

- **Ez a dia készíti elő a 3. szekciót.** Minden pont egy hiányosság amit az ESO+Vault megold.
- A "ki olvasta?" kérdés a legdrámaibb - audit hiánya.
- Ha kérdezik: az encryption at rest **nem oldja meg** a `kubectl get secret` problémát.

Slide: Alternatívák a natív Secret-en túl

- Sealed Secrets = "encrypted YAML Git-be teheted, klaszter visszafejti".
- SOPS = általánosabb, file-szintű (nem K8s-specifikus).
- A **last bullet (ESO)** átvezetés a 3. szekcióhoz - mondd ki: "erre épül a következő rész".

Slide: DEMO 2

- **Egyetlen fő üzenet:** `base64 -d` egy parancs, és minden titok plaintext-ben van.
- A `kubectl auth can-i` rész fontos - **a védelem RBAC-ben van, nem a Secret-ben**.
- Ne magyarázd túl - 2-3 perc bőven elég.

Időzítés - 2. rész

- ~10 perc szekció (2 perc demo + 8 perc dia).
- Ha csúszol: a "Secret típusok" táblázatot lehet átugrani.
- A "base64 ≠ titkosítás" diát **soha ne ugord át** - ez a fő tanulság.

---

3. rész: External Secrets Operator + Vault

Slide: Miért nem elég a natív Secret?

- A 2. szekció vége természetes átvezetés - itt **listázd a hiányosságokat**.
- A 6 pont fontossági sorrendben: nincs titkosítás → nincs audit → nincs rotáció.
- Hangsúlyozd: **éles környezet** - dev-ben a natív Secret oké.

Slide: Mi az a külső secret manager?

- A "dedikált rendszer" mondat a kulcs.
- Ne kezdj cloud provider-ek mély összehasonlításával.
- Egy mondat: "ha AWS-en vagy, valószínűleg Secrets Managert használsz - ha multi-cloud, Vault".

Slide: Két különálló réteg

- **Ezt a diát feltétlenül érdemes hangsúlyozni.** Sokan keverik a kettőt.
- "A tár és az integráció két különböző döntés."
- A demo Vault + ESO párost használja - de bármelyik réteg cserélhető.

Slide: Mi az az External Secrets Operator?

- "Operator" mint K8s pattern: CRD + reconciliation loop.
- A két CRD - `SecretStore` (kapcsolat) és `ExternalSecret` (mit-honnan-hova) - a fő mentális modell.
- A "transzparencia" rész fontos: a pod **nem tud Vault-ról**, csak egy K8s Secret-et lát.

Slide: Architektúra

- **Mutasd a diagramot, és kísérd narrációval:**
  1. Vault-ban él a titok.
  2. ESO API-n át lekéri.
  3. K8s Secret-et hoz létre/frissít.
  4. Pod a Secret-et mountolja.
- A "pod nem tud Vault-ról" gondolatot itt is mondd ki - ez biztonsági érv is.

Slide: Előnyök / Hátrányok

- A "dinamikus credentialek" pont **érdekes**: Vault tud időre szóló DB userset generálni.
- A "Hátrányok" oszlop a fontos: **lokális dev-re overkill**.
- Üzleti döntés: "van-e csapat, aki üzemelteti?" - Vault nem self-running.

Slide: Hogyan működik belül?

- A reconciliation loop koncepciója fontos - K8s Operator pattern alap.
- A `refreshInterval` trade-off: gyors frissítés vs Vault terhelés vs API rate limit.

Slide: SecretStore példa

- Kiemelendő: a `kubernetes` auth method - **a secret zero problémát ezzel oldjuk meg**.
- A `serviceAccountRef` az, ami a JWT-t adja a Vault-nak.

Slide: ExternalSecret példa

- A `refreshInterval: "15s"` a demo miatt rövid - prod-ban inkább `5m` vagy `15m`.
- A `data:` szekció **átnevezést** is enged: Vault `db-password` → K8s Secret `password`.

Slide: A "Secret zero" probléma

- **Ez a kedvenc filozófiai diám.** "Ha minden titok titkosítva van, mivel hitelesítünk?"
- A Kubernetes Auth megoldja: a SA JWT-t **a klaszter biztosítja**, nem mi tároljuk.
- Cloud-on Workload Identity még tisztább: nincs explicit credential.

Slide: Biztonság és limitációk

- A "network dependency" rész fontos: ha Vault meghal, **új pod-ok nem indulnak**.
- Mitigation: Vault HA + cache.
- A komplexitás trade-off: minden plusz komponens egy plusz failure mode.

Slide: Provider váltás - vendor-függetlenség

- **Wow-pont:** ugyanaz az `ExternalSecret`, csak a `SecretStore` provider-blokk változik.
- "Ha holnap AWS-re költözünk, a Secret-eket nem kell átírni."

Slide: Alternatív integrációk

- ESO vs Secrets Store CSI Driver: az **ESO K8s Secret-et generál**, a CSI **közvetlenül volume-ot mountol**.
- Vault Agent Sidecar: minden pod mellé sidecar - **több erőforrás, kevesebb függőség**.
- Sealed Secrets: másik világ - encrypted Secret Git-ben, nem külső tárral.

Slide: DEMO 3

- **A wow-pillanat: Vault-ban módosítasz egy értéket → 15 mp múlva a log-ban megjelenik az új érték, restart nélkül.**
- Két terminál szükséges:
  - 1. terminál: `kubectl logs -l app=demo-app -f`
  - 2. terminál: `kubectl exec ... vault kv put ...`
- Ha jut idő: a Vault UI is demoolható (`port-forward 8200`).

Időzítés - 3. rész

- ~15-18 perc szekció (5-6 perc demo + 10-12 perc dia).
- Ha csúszol: az "Alternatív integrációk" táblázatot lehet átugrani.
- Ha gyors vagy: a Vault UI-t is mutasd be élőben.

Demo előtti checklist

- [ ] `setup.sh` egyszer már lefutott helyben (vagy kéznél van)?
- [ ] Két terminál nyitva, fontméret nagyra állítva?
- [ ] `kind` és `helm` PATH-ban van?
- [ ] Vault UI port-forward parancs előkészítve copy-paste-re?
- [ ] Backup terv: ha valami nem megy, a `DEMO.pdf` screenshot-jai segítenek.

Teljes előadás időzítés

- 1. rész (ConfigMap): ~10-12 perc
- 2. rész (Secret): ~10 perc
- 3. rész (ESO+Vault): ~15-18 perc
- **Összesen: ~35-40 perc**